

Presentation Script: ICIC 2026

Paper Title: Benchmarking Multi-View Tree-Level Oil Palm Bunch Counting Under a Fixed Detector

Presenter: Muhammad Zainal Muttaqin

Affiliation: Lambung Mangkurat University, Banjarbaru, Indonesia

Target Duration: ~8 minutes speaking + 2 minutes Q&A buffer

Slide Deck: 057_MUTTAQIN.pptx (13 slides)

How to read: *Idea* is the one thing the audience should understand on that slide. *[Brackets]* mark an action, not speech.

Slide 1: Title

Idea: the whole talk is about one question. When the count is wrong, is it the detector or the counter?

Good morning, everyone. My name is Muhammad Zainal Muttaqin, and I am from Lambung Mangkurat University.

Today I will present our paper, titled "Benchmarking Multi-View Tree-Level Oil Palm Bunch Counting Under a Fixed Detector".

In simple words, this work is about counting oil palm fruit bunches from photos. A system like this usually has two parts: one model that finds the bunches in each photo, and another one that turns those detections into a final count for the tree. So the question I want to answer today is quite simple. When the final count is wrong, which of the two is actually responsible?

Slide 2: Black Bunch Census: four maturity classes

Idea: the plantation needs four numbers per tree, one for each maturity class.

Let me start with why we count bunches in the first place. In an oil palm plantation, the harvest has to be planned ahead, and to do that the manager needs to know how many bunches are on each tree and how ripe they are.

This survey is called the Black Bunch Census. Every bunch on the tree is put into one of four classes, which you can see here. B1 is the ripe one, B2 is starting to turn red, and B3 and B4 are still unripe, so they will be harvested later.

Right now, this is done manually. Workers walk from tree to tree and count by eye, which takes a lot of time and also depends on who is counting. That is why we want to do it with a camera.

Slide 3: How we collected SawitMVC

Idea: we photograph each tree from four sides, and the experts tell us which boxes are the same bunch.

To study this, we built our own dataset, which we call SawitMVC. We collected it in a field survey at two plantations in Indonesia, DAMIMAS and LONSUM, using a smartphone.

The problem with taking only one photo is that you cannot see the whole tree. The bunches grow all around the trunk, and some of them are hidden behind the leaves. So for each tree, we walked around it and took photos from four sides, and for a small number of trees from eight sides. In total we have 953 trees and almost four thousand images.

After that, domain experts drew a box around every bunch and gave it a class. And there is one more step, which is very important for us: they also marked which boxes in different photos belong to the same physical bunch. Because of that, we know the true number of bunches on every tree.

Slide 4: One bunch appears in several views

Idea: the same bunch shows up in several photos, so adding up the boxes counts it more than once.

Now, taking photos from several sides solves one problem, but it creates another one.

Please look at the bunch inside the orange box. *[point]* You can see it in the first view, then again in the second view, and again in the third view. So we have three boxes, but in reality it is only one bunch.

And this happens a lot. On average, one bunch appears about 1.89 times, so almost twice. That means if we just add up all the boxes, we will get almost double the real number.

So we need something that takes the boxes from all the photos and turns them into the real number of bunches on the tree. In this talk, I will call that part the counter.

Slide 5: Ground truth vs fixed detector

Idea: we give the same counter two kinds of boxes. Whatever accuracy we lose between them comes from detection errors.

Of course, in a real system nobody draws the boxes by hand. A detector does that, and a detector makes mistakes. So when the final count is wrong, we do not know if the problem comes from the detector or from the counter.

To separate the two, we run every counter twice.

The first time, on the left, we give it the ground truth boxes, the ones drawn by the experts. Here every bunch is found and every class is correct.

The second time, on the right, we give it the boxes from a YOLO26m detector. We trained this detector once on the 716 training trees and then we did not change it anymore, which is why we call it a fixed detector. And as you can see on this tree from the test set, it is not perfect. It misses one bunch completely, it labels a B2 bunch as B3, and it puts a second box on a bunch it has already found.

The important point is that the counting method is the same in both cases, and the only thing that changes is the boxes. So if the accuracy drops, we know that the drop comes from the detection errors.

Slide 6: Step 1: boxes to tree features

Idea: the counter does not look at the photos. It looks at a few simple numbers that summarise the boxes of one tree.

So how does a counter actually work? It does not look at the images at all. It only looks at a few numbers that summarise the boxes of one tree.

Let me use this example for class B3. This tree has four views, and each view contains two B3 boxes. *[point: the yellow boxes]* From these, we compute three simple numbers. The sum is eight. The maximum in a single view is two. And the mean per view is two.

Why these three? Because the sum is usually too high, since it counts the same bunch several times. And the maximum is usually too low, since one photo cannot see every bunch. On this tree the true number of B3 bunches is four, so the real answer is somewhere in between, and these numbers help the counter find it.

We do the same for all four classes, and we add the number of views. That gives us thirteen numbers for each tree, and this is our baseline feature set. We also tried a larger set with up to 67 features, and I will come back to that later.

Slide 7: Step 2a: counting without machine learning

Idea: the simplest counters are fixed rules. Divide the sum by how often a bunch repeats on average.

Now, the counting itself. We start with two very simple rules that do not need any training.

The first one is the naive sum. It just takes the total, which is eight in our example, and as we already saw, that counts the same bunch several times.

The second one is a bit smarter. We know that on average one bunch appears about 1.89 times, so we simply divide the sum by that number. In our example, eight divided by 1.89 gives about four. We get this divisor from the training trees only, and we compute it separately for each kind of boxes: it is 1.89 for the ground truth boxes and 1.79 for the detector boxes.

Slide 8: Step 2b: counting with machine learning

Idea: five standard regression models. We want to see how much the choice of counter really matters.

But using one divisor for all trees is a rough rule, because different trees can have different patterns. So we also train regression models on the 716 training trees. Each model takes the features of one tree as input, thirteen numbers in the baseline, and gives four numbers as output, which are the counts for B1 to B4.

We compare five models that are all quite standard: linear regression, Ridge, ElasticNet, SVM, and random forest. We did not try to design a new counter here. What we want to see is how much the choice of counter really matters.

Slide 9: Evaluation: accuracy within ± 1 bunch

Idea: Class ± 1 checks each class separately. Tree ± 1 is stricter, because all four classes must be right together.

Before I show the results, let me explain how we measure accuracy, because we use two metrics and they are easy to mix up.

In both of them, we say a prediction is correct if it is within one bunch of the true count. So if a tree has five bunches and we predict four or six, we still count that as correct.

Let me show you with this example tree. For B1, the true count is one and we predict one, so that is correct. For B2, it is two and we predict three, which is still within one. For B4, it is two and two, also correct. But for B3, the true count is five and we predict seven. That is off by two, so it is wrong. *[point: B3 column]*

The first metric is Class ± 1 . It looks at every class separately, so this tree gets three out of four, or 75 percent.

The second metric is Tree ± 1 , and it is much stricter. Here the tree is only counted as correct if all four classes are correct at the same time. Because B3 is wrong, the whole tree is counted as wrong, so it gets zero.

So when you see the results, please remember that the tree-level number can never be higher than the class-level number.

Slide 10: Results: ground truth vs fixed detector

Idea: changing the counter changes the result by about 3 points. Changing the boxes changes it by about 21.

Now let us look at the results on the 141 test trees.

First, the green bars. These are the counters when we give them the ground truth boxes. The naive sum on the left only gets 50 percent, which is what we expected. But all the other counters are very high, around 96 to 98 percent. And if we use the strict metric, the best counter still gets 92 percent of the trees right in all four classes. So when the boxes are correct, counting from several views is actually not that difficult, even with simple models.

Now look at the orange bars. These are the same counting methods, but this time with the boxes from the detector. The accuracy goes down a lot. The best bar in this chart is about 76 percent, and even our best configuration overall only reaches 77 percent. With the strict metric, only about one tree in three is right in all four classes.

And there is one more thing I want you to notice. If you compare the five machine learning counters with each other, the difference between the best and the worst is only about three points, and that difference is not statistically significant. So changing the counter gives us about three points, but changing the boxes costs us about 21 points. That is the main result of this paper.

Slide 11: Detection errors behind the gap

Idea: the detector misses many bunches and confuses B2 with B3. The counter cannot fix what it never receives.

So the next question is, what exactly goes wrong in the detector? The chart on the left gives the overall picture. Out of all the bunch appearances in the test images, the detector finds and labels correctly only about 48 percent. About 37 percent are missed, and about 15 percent are found but given the wrong class. So there are two main problems.

The first one is missed bunches. Some of them are still caught in another view, but about 23 percent of the bunches are never detected in any of the views. And for B4, the small and very unripe bunches, it is even worse, more than 40 percent. If a bunch is never detected, the counter has no information about it at all, so the best it can do is guess.

The second problem is the wrong class. Sometimes the detector finds the bunch but gives it the wrong label. The most common case is B2 being labelled as B3, which happened 158 times in the test set. In that case the total number of bunches is still right, but the bunch is counted in the wrong class.

We also asked ourselves if a better counter could fix this. So we gave the counter more information, 67 features instead of 13. But the improvement was only 1.4 points, and it was not statistically significant.

Finally, we wanted to make sure this is not just a coincidence from one setup. So we repeated the comparison in four ways: with the configuration chosen on the validation set, with cross-validation, with a different detector, YOLO11m, and with four YOLO26 checkpoints. In all of these checks, the gap stays between 23 and 25 points.

Slide 12: Takeaways

Idea: on this dataset, the detector matters much more than the counter, so that is where to improve first.

So let me come back to the question from the beginning. When the count is wrong, is it the detector or the counter?

On our dataset, it is mostly the detector. We lose about 21 points because of the detector, and only about three points between the different counters.

This also tells us where to work next. The detector needs to find more of the B3 and B4 bunches, and it needs to be better at telling B2 and B3 apart.

We also have one suggestion for other people who work on counting. Please report your counter with both the ground truth boxes and the detector boxes, per class and per tree. If you only use one of them, you cannot really tell where the errors come from.

I should also be honest about the limits. This is one dataset from two estates, and the ground truth result is an empirical reference for us, not an upper bound. For future work, we plan to try explicit multi-view association, to repeat the detector training, and to test on an independent plantation.

Slide 13: Thank you

That is all from me. Our code and results are available on GitHub, and the link is on the slide. Thank you very much for listening, and I would be happy to answer your questions.

Anticipated Questions & Answers (Cheat Sheet)

Q: What exactly are the features? Can you list them?

A: Sure. The baseline has thirteen features. For each of the four classes we take three numbers: the total detections over all views, the maximum in a single view, and the mean per view. That is twelve, and the last one is the number of views. The extended set has 67 features. On top of the baseline, it adds four groups. The first is confidence: the sum, mean and maximum confidence for each class, and how many boxes are above 0.5 and above 0.6. The second is position and size: the average vertical position of the boxes and their average area. The third is the spread across views: the standard deviation, the minimum, the coefficient of variation, how many views contain the class, and a consistency score. And the last one is composition: the total number of detections, the share of each class, and the share of B3 among B2 and B3.

(Quick count if asked: 13 baseline + 20 confidence + 8 position and size + 20 spread + 6 composition = 67.)

Q: Is harvest planning in real plantations based only on the Black Bunch Census?

A: No, not only that. The daily harvest is decided by the harvesters in the field, who check the ripe bunches and the loose fruit on each round, typically every 7 to 10 days. The Black Bunch Census is a periodic survey, usually every three or four months, and it is mainly used to forecast the crop one to four months ahead and to plan labor and transport. Our work is aimed at automating that census.

Q: Why not use 3D reconstruction or tracking to match bunches across views?

A: That is a good question. Those methods usually need a lot of overlap between images, a moving sensor, or calibrated cameras. In our case we only have four to eight smartphone photos per tree, taken from different sides. So we started with a simpler approach, which is regression on summary features. Comparing it with explicit matching on the same inputs is the next thing we want to do.

Q: Is ground truth an upper bound for counting accuracy?

A: Not really. We treat it as an empirical reference. It shows what these counters can reach when the boxes and classes are all correct. A different counter in the future could use information that ours do not use, so we do not want to call it an upper bound.

Q: Why does Random Forest perform worse than the linear counters?

A: A random forest makes its prediction by averaging similar training examples, so it tends to pull high counts down towards the more common values. For example, for B1 it never predicts more than three bunches, even though some trees have five or six. I should add that with detector boxes, this difference is no longer statistically significant.

Q: Would a larger or newer detector close the gap?

A: Not in the experiments we did. With YOLO11m the gap is about 24 points, and with four YOLO26 checkpoints it is about 23. But I have to be careful here, because these are all YOLO models at a similar level, around 0.52 mAP50 for YOLO26m, and YOLO11m was only trained once. A much stronger detector should make the gap smaller, and that is exactly where we suggest putting the effort.

Q: Why not lower the confidence threshold to recover the missed bunches?

A: We tried that. We tested eleven thresholds from 0.05 to 0.70. A lower threshold does find more bunches, but it also adds a lot of false boxes, and none of the thresholds gives more than 77.5 percent at class level. At 0.10 the tree-level accuracy goes up from 33 to 38 percent, but we ran this on the test set, so we only report it as an exploratory result.

Q: Why is B3 the hardest class to count, even though its recall is higher than B2 and B4?

A: There are two reasons. First, B3 is the largest class, more than half of all bunch appearances in the test set, so a tree usually has more B3 bunches and it is easier to be off by two. Second, B3 also receives the B2 bunches that the detector labels wrongly. So recall alone does not tell us which class is hardest to count.

Q: Why a tolerance of one bunch? What about other metrics?

A: The counts per class are small, around two to three bunches per tree on average, so one bunch is the smallest tolerance after an exact match. In the paper we also report MAE, RMSE and bias, and they show the same pattern. The mean error per class goes from about 0.3 bunches with ground truth boxes to about 1.0 with detector boxes.

Q: What exactly is the "fixed detector", and why did you keep it fixed?

A: It is a single YOLO26m model that we trained once on the 716 training trees and then froze, with a confidence threshold of 0.25. Because it is fixed, every counter receives exactly the same boxes. So if two counters give different results, we know the difference does not come from the detector.

Q: How well do these results generalize to other plantations?

A: We cannot claim that yet. The data come from two estates in South Kalimantan, and the smaller one only has 99 trees. In our cross-estate test, the effect of the estate is mixed with the effect of the training set size, and the detector has seen both estates. So testing on an independent plantation is part of our future work.